Hochschule für Technik, Wirtschaft und Kultur Leipzig
University of Applied Sciences
Fakultät Elektrotechnik und Informationstechnik

MODUL 5420 - EMBEDDED SYSTEMS I

Project 1: Real-Time Environmental Sensor Node

Beleg & Dokumentation

Prjektteam: Zakaria Rajjaa
Jamal Elouartnani
Fatima Sadeq

Betreuer: Prof. Dr.-Ing. Andreas Pretschner

17. März 2026

Inhaltsverzeichnis

Abbildungsverzeichnis i
Tabellenverzeichnis i
1. Einleitung 1
1.1 Motivation und Zielsetzung 1
1.2 Funktionsweise des entwickelten Systems 2
2. Planungsphase 3
2.1 Projektplan und Zeitmanagement 3
2.2 Aufgabenübersicht 3
3. Anforderungsanalyse und verwendete Komponenten 4
3.1 Anwendungsbereiche 4
3.2 Übersicht und Beschreibung der Hardwarekomponenten 4
3.3 Verwendete Softwarekomponenten 5
4. Systemarchitektur und Design 6
4.1 Aufbau und Struktur 6
4.2 Blockdiagramm des Systems 6
5. Hardwareimplementierung 7
5.1 Verwendete Hardware und PinZuweisung.....7
5.2 Verkabelungsschema 8
5.3 Praktischer Hardwareaufbau 9
6. Softwareentwicklung und Implementierung 10
6.1 Entwicklungsumgebung (PlatformIO) 10
6.2 Konfiguration der PlatformIO-Umgebung 10
6.3 Bibliotheken, Pin-Definitionen und Setup 11
7. Hauptfunktionen des Codes 14
7.1 Task 1: Sensorerfassung (Acquisition) 14
7.2 Task 2: Datenverarbeitung (Processing).....15
7.3 Task 3: Kommunikation (Communication) 17
7.4 Task 4: Überwachung (Supervisor) 19
8. Tests und Validierung 21
8.1 Serielle Monitor-Ausgabe 21
8.2 Fehleranalyse und Lösungsansätze.....23
8.3 Ergebnisse und Diskussion 24
9. Fazit 25

Abbildungsverzeichnis

Abbildung 1: Prozessablauf FreeRTOS-Task-Kette 2
Abbildung 2: Blockdiagramm des Systems 6
Abbildung 3: Verkabelungsschema Arduino Mega 2560 8
Abbildung 4: Praktischer Hardwareaufbau..... 9

Tabellenverzeichnis

Tabelle 1: Komponentenliste 4
Tabelle 2: Pinbelegung 7
Tabelle 3: RTOS-Objekte 13
Tabelle 4: Task-Übersicht 13
Tabelle 5: Alarm-Schwellwerte 16
Tabelle 6: Funktionstests 22
Tabelle 7: Deadline-Analyse und CPU-Auslastung 24

1. Einleitung

1.1 Motivation und Zielsetzung

• Zielsetzung:

Das Hauptziel dieses Projekts ist die Entwicklung eines lauffähigen, modularen und nachvollziehbaren eingebetteten Echtzeitsystems, das Methoden des Echtzeit-Schedulings und der Inter-Task-Kommunikation korrekt anwendet. Das System soll in der Lage sein, kontinuierlich Umweltparameter zu messen, diese mittels eines gleitenden Mittelwerts zu filtern und bei Überschreitung definierter Schwellwerte einen Alarm auszulösen – sowohl auf dem Display, durch LEDs, als auch akustisch durch einen Buzzer.

Der Anwendungsbereich umfasst die Umweltüberwachung in Innenräumen, beispielsweise zur Überwachung der Luftqualität in Lager- oder Serverräumen. Das System arbeitet autark ohne Serververbindung oder Cloud-Anbindung.

Ein weiteres Ziel des Projekts ist die modulare und skalierbare Entwicklung, sodass das System problemlos an unterschiedliche Anforderungen angepasst oder um weitere Funktionen erweitert werden kann.

• Themenüberblick und Motivation

Eingebettete Systeme sind ein zentraler Bestandteil moderner Automatisierungstechnik. Sie ermöglichen es, Sensordaten effizient zu erfassen und in Echtzeit zu verarbeiten. In diesem Projekt wurde ein System entwickelt, das Temperatur-, Feuchtigkeits- und Luftqualitätsmessungen durchführt und die Werte auf einem LCD-Display anzeigt. Zusätzlich werden Alarme durch LEDs und einen passiven Buzzer signalisiert.

Technisch betrachtet setzt das System auf eine Kombination mehrerer Module:

- DHT11-Sensor über 1-Wire-Protokoll zur Erfassung von Temperatur und Luftfeuchtigkeit
- MQ-135-Sensor (ADC) zur Erfassung der Luftqualität (CO_2, NH_3, Benzol)
- LCD 1602 (I2C, PCF8574) für die visuelle Ausgabe von Messwerten und Alarmstatus
- FreeRTOS mit vier Tasks, Queues und Event Groups für echtzeitfähiges Multitasking
- Status-LEDs und passiver Buzzer als optische und akustische Alarmsignale

Durch die erfolgreiche Umsetzung dieses Projekts konnten tiefgehende Kenntnisse in Mikrocontroller-Programmierung, Echtzeitbetriebssystemen und Multitasking erlangt werden. Besonders herausfordernd war die Realisierung einer effizienten Lösung mit begrenzten Hardware-Ressourcen des ATmega2560.

1.2 Funktionsweise des entwickelten Systems

Das System erfasst kontinuierlich Temperatur, Luftfeuchtigkeit und Luftqualität. Die Rohdaten werden gefiltert (gleitender Mittelwert über 5 Messwerte) und mit definierten Schwellwerten verglichen. Bei Überschreitung wird ein Alarm ausgelöst – über LED, Buzzer und LCD-Display. Vier FreeRTOS-Tasks arbeiten dabei parallel und kommunizieren über Queues und Event Groups.

Der nachfolgende Prozessablauf veranschaulicht die Funktionsweise des Systems detailliert:

Task	Aktion
Acquisition (Prio 3 · 2000 ms)	DHT11 + MQ-135 lesen · NaN-Retry · Heartbeat-LED · xQueueRawData senden · EVT_ACQ_ALIVE
Processing (Prio 2 · event-driven)	Ringpuffer (5) · gleitender Mittelwert · Schwellwertprüfung · Alarm-LEDs + Buzzer · EVT_PROC_ALIVE
Communication (Prio 1 · event-driven)	LCD-Ausgabe (I2C 0x27) · UART CSV-Ausgabe · EVT_COMM_ALIVE setzen
Supervisor (Prio 4 · 5000 ms)	xEventAlive prüfen · bei fehlendem Signal → Fehler-LED (Pin 13) AN

Abbildung 1: Prozessablauf der FreeRTOS-Task-Kette

2. Planungsphase

2.1 Projektplan und Zeitmanagement

• **Projektzeitplan:**

Dieser Abschnitt beschreibt die Phasen der Entwicklung, von der Anforderungsanalyse bis zur Integration und Dokumentation.

Phase	Beschreibung	Dauer
Anforderungsanalyse	Festlegung der Anforderungen für Sensoren, FreeRTOS, Alarme	1 Woche
Hardware-Planung	Auswahl der Bauteile (DHT11, MQ-135, LCD, LEDs, Buzzer, Arduino Mega)	1 Woche
Schaltplan-Design	Erstellen des Verkabelungsplans und Pin- Zuweisung aller Komponenten	1 Woche
Software-Entwicklung	Implementierung der FreeRTOS-Tasks, Queues, Event Groups	2 Wochen
Integration & Tests	Hardware- und Software-Integration, Fehlerbehebung & Optimierung	1 Woche
Dokumentation	Erstellung einer technischen Dokumentation	1 Woche

2.2 Aufgabenübersicht

In diesem Abschnitt werden die Hauptaufgaben innerhalb des Projekts detailliert beschrieben.

Aufgabe	Beschreibung
Schaltplan erstellen	Zeichnen des elektronischen Aufbaus mit Verbindungen zwischen den Bauteilen
Hardware-Komponenten vorbereiten	Anschließen von DHT11, MQ-135, LCD (I2C), LEDs und Buzzer an den Arduino Mega 2560
Entwicklung des Codes	Implementierung der FreeRTOS-Logik mit vier Tasks, Queues und Event Groups
Testing & Debugging	Durchführung von Tests zur Überprüfung der korrekten Funktionsweise aller Module
Optimierung & Fehlerbehebung	Verbesserung des Codes für stabilere Sensordatenerfassung und zuverlässige Alarme
Erstellung der Dokumentation	Technische Dokumentation mit Beschreibung der Architektur, Schaltpläne, Code und Testprotokolle

Mit diesen Aufgaben und Zeitplänen wird sichergestellt, dass das Projekt strukturiert und effizient durchgeführt wird.

3. Anforderungsanalyse und verwendete Komponenten

3.1 Anwendungsbereiche

Das entwickelte Umgebungsüberwachungssystem eignet sich für private, kommerzielle und industrielle Einsatzgebiete, in denen Luftqualität, Temperatur und Feuchtigkeit relevant sind. Beispiele sind Serverräume, Lagerräume, Bürogebäude, Labore oder Wohnbereiche.

3.2 Übersicht und Beschreibung der Hardwarekomponenten

Die folgende Tabelle zeigt die für das Projekt verwendeten Hardwarekomponenten und deren Funktionen:

Bauteil	Beschreibung
Arduino Mega 2560	Mikrocontroller (ATmega2560, 16 MHz, 8 KB RAM) zur Steuerung aller Komponenten
DHT11 Sensor	Digitaler Sensor an Pin 2 für Temperatur (±2 °C) und Luftfeuchtigkeit (±5 %)
MQ-135 Sensor	Analoger Gassensor (CO_2, NH_3, Benzol) an A0, gibt ADC-Wert 0–1023 aus
LCD 1602 + PCF8574	16×2-Zeichen-Display mit I2C-Adapter (Adresse 0x27), SDA=Pin 20, SCL=Pin 21
LED grün + 220 Ω	Heartbeat-LED an Pin 7 – zeigt laufenden Betrieb alle 2 s
LED rot + 220 Ω	Temperatur-Alarm-LED an Pin 8
LED gelb + 220 Ω	Feuchtigkeits-Alarm-LED an Pin 9
LED blau + 220 Ω	Luftqualitäts-Alarm-LED an Pin 10
Passiver Buzzer	An Pin 11 über PN2222-NPN-Transistor (1 kΩ Basiswiderstand), 1000 Hz Ton bei Alarm
On-Board LED	LED an Pin 13 – leuchtet bei fehlendem Task-Alive-Signal (Supervisor-Fehler)
Breadboard + Kabel	Zur Verdrahtung aller Komponenten

Tabelle 1: Komponentenliste

3.3 Verwendete Softwarekomponenten

Neben der Hardware ist auch die richtige Softwareumgebung entscheidend für das Projekt.

Software	Verwendungszweck
PlatformIO (VSCode)	Hauptentwicklungsumgebung zur Programmierung des Arduino Mega
Arduino IDE	Alternative IDE für schnelle Tests und Debugging
FreeRTOS (feilipu)	Echtzeitbetriebssystem für AVR – Tasks, Queues, Event Groups, Scheduler
Adafruit DHT @^1.4.6	Treiberbibliothek für den DHT11-Sensor
LiquidCrystal_I2C @^1.1.4	Bibliothek zur Ansteuerung des LCD 1602 über I2C (PCF8574)
GitHub/Git	Versionskontrolle und Speicherung des Codes
Serial Monitor	Debugging und Echtzeitüberwachung der CSV-Ausgabe

4. Systemarchitektur und Design

4.1 Aufbau und Struktur

Das System kombiniert Sensoren und Aktoren, um eine zuverlässige Umgebungsüberwachung zu gewährleisten. Der Arduino Mega 2560 verarbeitet Eingaben von DHT11 und MQ-135 über FreeRTOS-Tasks und steuert LCD-Display, Alarm-LEDs und Buzzer als Ausgabe.

Die Implementierung basiert auf vier unabhängigen FreeRTOS-Tasks, die über zwei Queues und eine Event Group miteinander kommunizieren. Die modulare Architektur mit getrennten Dateien in src/ und include/ ermöglicht eine einfache Wartung und Erweiterbarkeit.

4.2 Blockdiagramm des Systems

Das Blockdiagramm zeigt den Prozessablauf:

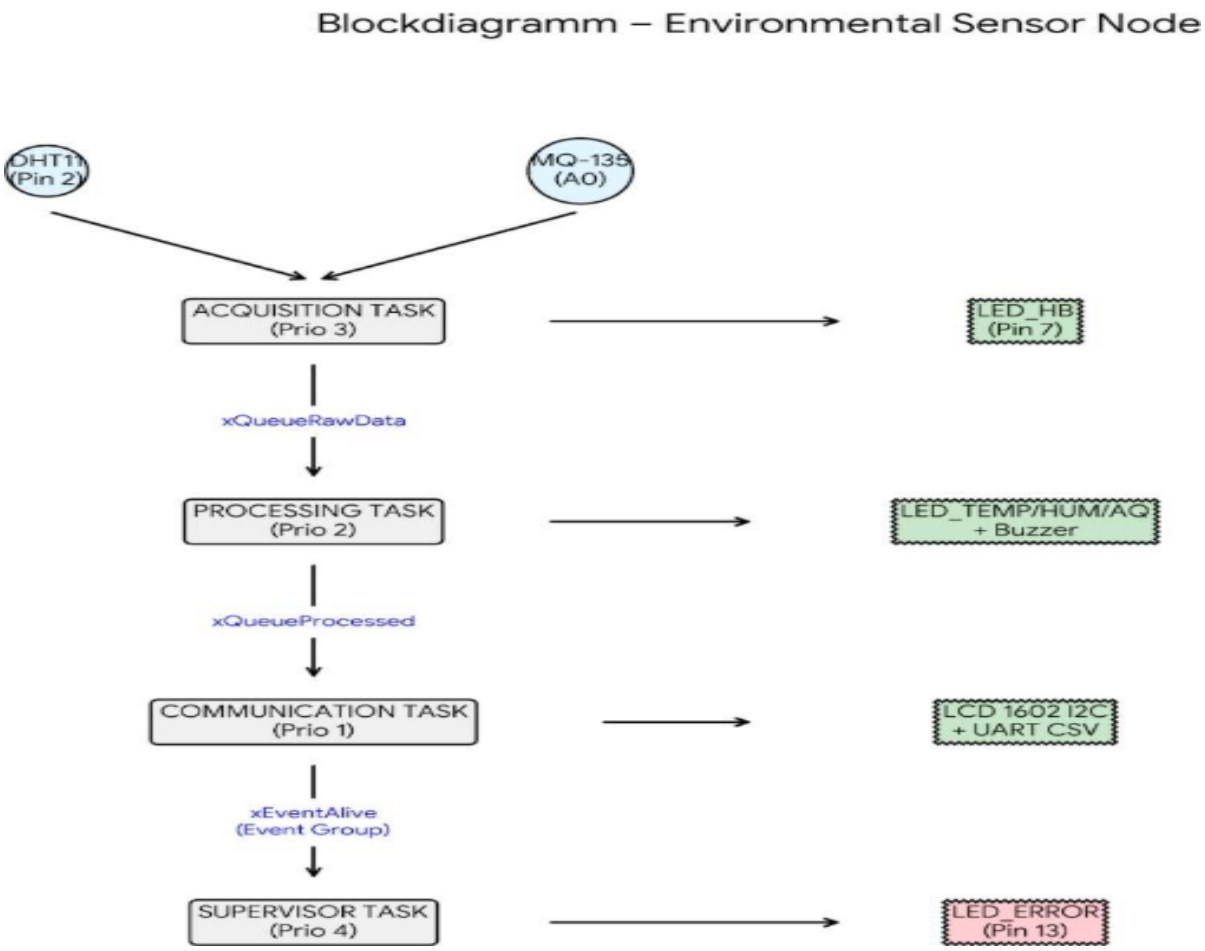

Abbildung 2: Blockdiagramm des Systems

5. Hardwareimplementierung

5.1 Verwendete Hardware und Pin-Zuweisung

Die folgende Tabelle zeigt die verwendeten Hardware-Komponenten und deren Pin-Zuweisung zum Arduino Mega 2560.

Bauteil	Signal / Pin	Typ	Arduino Mega 2560 Pin
DHT11	VCC	5V	Arduino 5V
	GND	–	Arduino GND
	DATA	Digital	Pin 2
MQ-135	VCC	5V	Arduino 5V
	GND	–	Arduino GND
	AOUT	ADC	Pin A0
LCD 1602 (I2C)	VCC	5V	Arduino 5V
	GND	–	Arduino GND
	SDA	I2C	Pin 20
	SCL	I2C	Pin 21
LED grün + 220 Ω	→ GND	GPIO	Pin 7
LED rot + 220 Ω	→ GND	GPIO	Pin 8
LED gelb + 220 Ω	→ GND	GPIO	Pin 9
LED blau + 220 Ω	→ GND	GPIO	Pin 10
On-Board LED	–	GPIO	Pin 13
Buzzer via PN2222	1 kΩ Basis	GPIO	Pin 11
	Kollektor → Buzzer (–)	–	–
	5V → Buzzer(+)	5V	5V

Tabelle 2: Pinbelegung

5.2 Verkabelungsschema

Die Schaltung zeigt die Verkabelung des Arduino Mega 2560 mit DHT11, MQ-135, LCD-Display (I2C), Status-LEDs und passivem Buzzer.

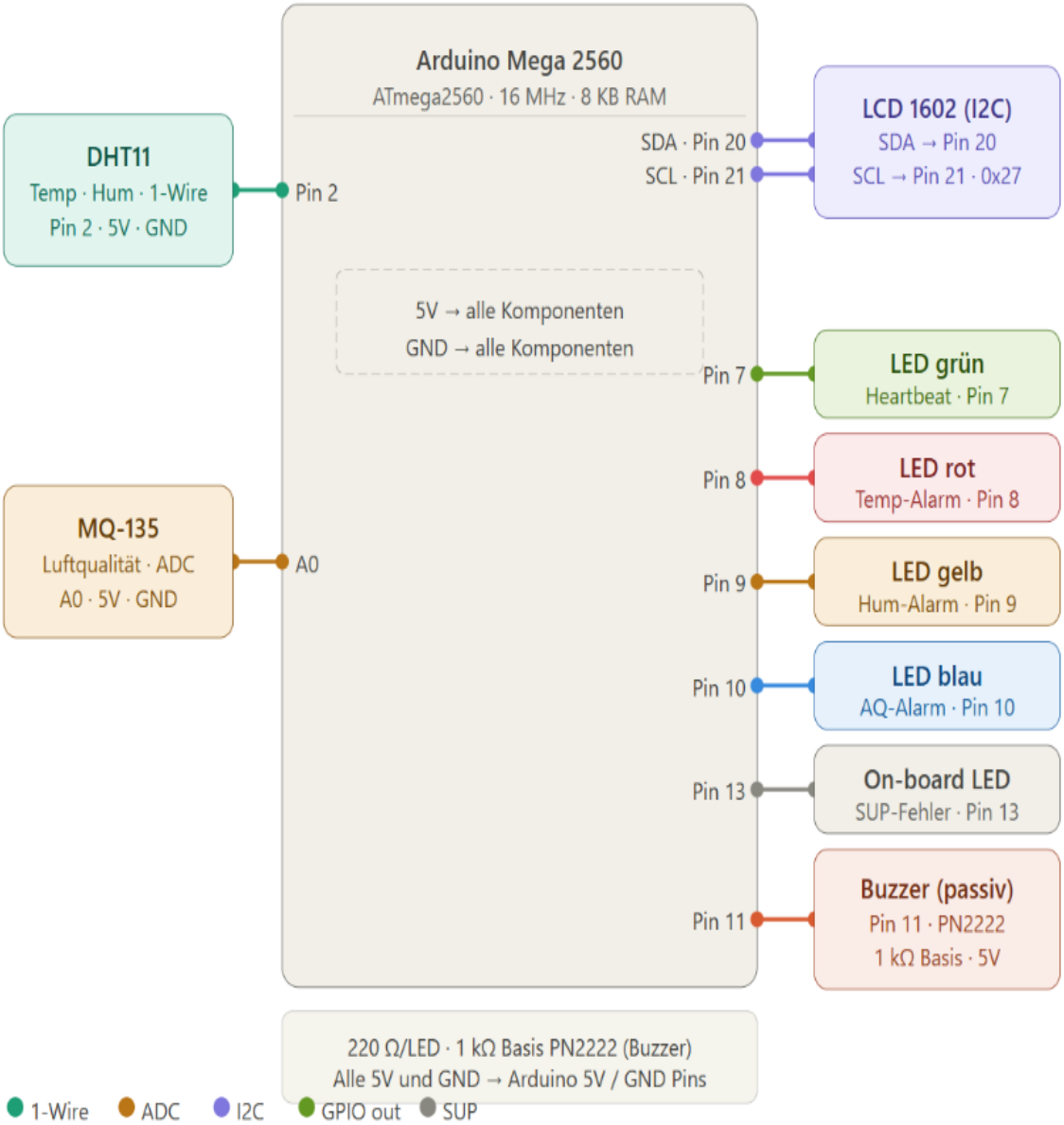

Abbildung 3: Verkabelungsschema Arduino Mega 2560 mit Sensoren, LCD, LEDs und Buzzer

5.3 Praktischer Hardwareaufbau

Das System wurde auf einem Breadboard aufgebaut. Alle Komponenten – DHT11, MQ-135, LCD 1602, Status-LEDs und passiver Buzzer – sind über Jumper-Kabel mit dem Arduino Mega 2560 verbunden. Die Installation von PlatformIO erfolgte als Extension in Visual Studio Code. Nach dem Öffnen des Projektordners werden alle Libraries automatisch über die platformio.ini heruntergeladen und kompiliert.

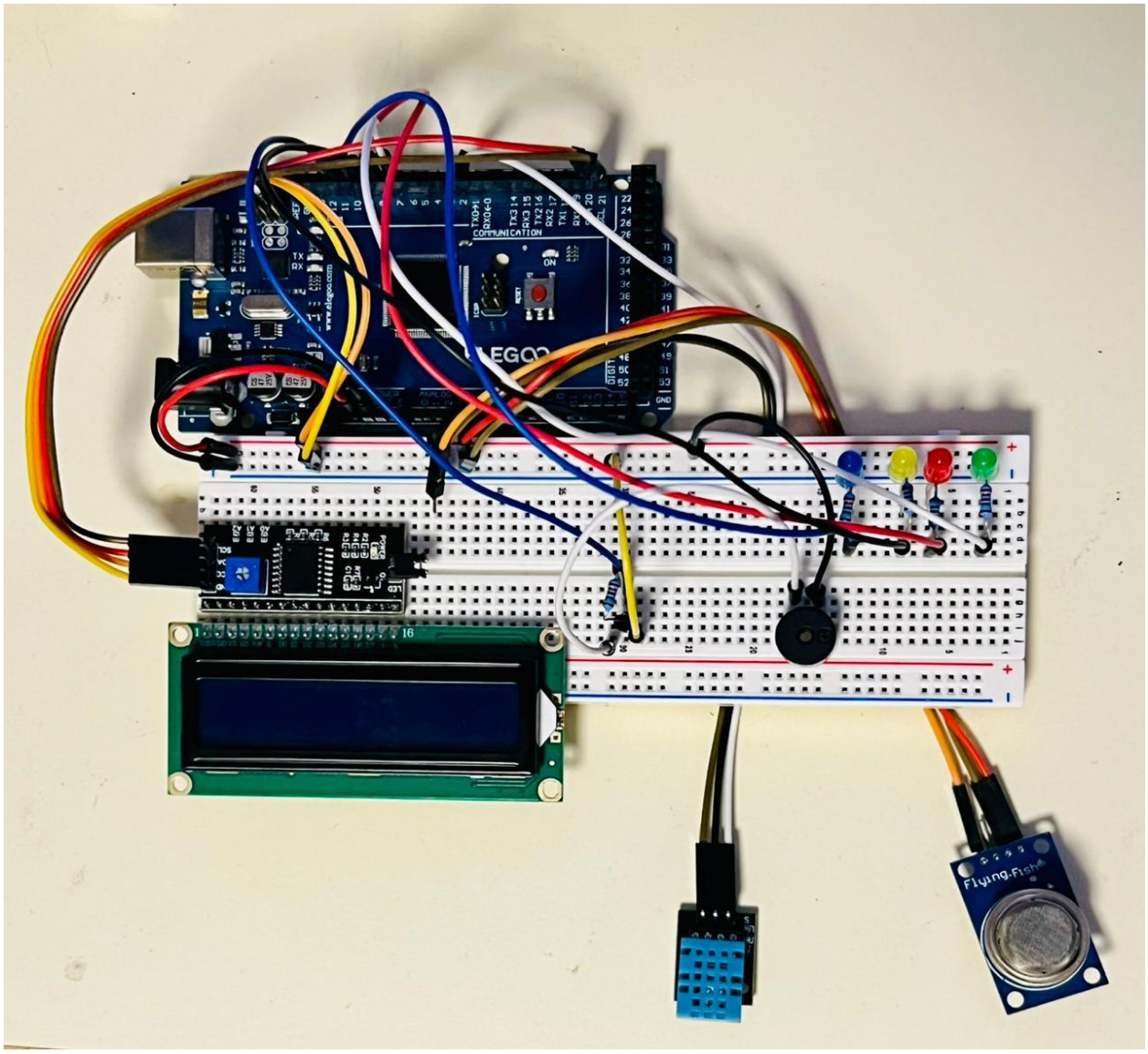

Abbildung 4: PraktischerHardwareaufbau

6. Softwareentwicklung und Implementierung

6.1 Entwicklungsumgebung (PlatformIO)

Die Programmierung wurde in PlatformIO innerhalb von Visual Studio Code durchgeführt. PlatformIO bietet eine erweiterte Entwicklungsumgebung mit einer besseren Bibliotheksverwaltung und einer optimierten Build-Umgebung für den Arduino Mega 2560.

6.2 Konfiguration der PlatformIO-Umgebung

Die Datei platformio.ini wurde entsprechend konfiguriert, um alle benötigten Bibliotheken einzubinden. Die wichtigsten Einstellungen umfassen:

- Board: megaatmega2560
- Framework: Arduino
- Monitor Speed: 115200 Baud
- Bibliotheken: FreeRTOS, DHT sensor library, LiquidCrystal_I2C

Die folgende platformio.ini Datei zeigt die vollständige Konfiguration:

platformio.ini

```
; ==============================================================================
;  PlatformIO – Arduino Mega 2560
;  Sensoren : DHT11 (Temp + Hum)  +  MQ-135 (Luftqualität)
;  Display  : LCD 1602 via I2C (PCF8574 Backpack, Adresse 0x27 oder 0x3F)
; ==============================================================================

[env:mega2560]
platform  = atmelavr
board     = megaatmega2560
framework = arduino

lib_deps =
    https://github.com/feilipu/Arduino_FreeRTOS_Library.git
    adafruit/DHT sensor library@^1.4.6
    adafruit/Adafruit Unified Sensor@^1.1.14
    marcoschwartz/LiquidCrystal_I2C@^1.1.4   ; I2C-LCD (PCF8574)

monitor_speed = 115200
```

Quellcode: platformio.ini Konfiguration

6.3 Bibliotheken, Pin-Definitionen und Setup

Dieses Code-Segment definiert die Pin-Verbindungen, initialisiert die Sensor-, LCD- und FreeRTOS-Module und legt die Schwellwerte sowie die RTOS-Objekte für die Task-Kommunikation fest.

```
#include <Arduino.h>

#include <Arduino_FreeRTOS.h>

#include <queue.h>

#include <event_groups.h>

#include <Wire.h>

#include <DHT.h>

#include <LiquidCrystal_I2C.h>

// Pin-Definitionen

#define DHT_PIN          2     // DHT11 Datenpin

#define MQ135_PIN        A0    // MQ-135 Analogeingang

#define PIN_LED_ERROR    13    // On-Board Status-LED (SUP-Fehler)

#define PIN_LED_HB       7     // Heartbeat-LED (grün)

#define PIN_LED_TEMP     8     // Temperatur-Alarm-LED (rot)

#define PIN_LED_HUM      9     // Feuchtigkeits-Alarm-LED (gelb)

#define PIN_LED_AQ       10    // Luftqualität-Alarm-LED (Blau)

#define PIN_BUZZER       11    // Passiver Buzzer via PN2222

// I2C-Adresse LCD

#define LCD_ADDR         0x27

// Schwellwerte

#define THRESHOLD_TEMP_HIGH   30.0f   // Grad C

#define THRESHOLD_HUM_HIGH    80.0f   // %RH

#define THRESHOLD_AQ_HIGH     600     // ADC-Wert (0-1023)
```

Quellcode: Bibliotheken und Pin-Definitionen

```
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <queue.h>
#include <event_groups.h>

#include "task_acquisition.h"
#include "task_processing.h"
#include "task_comm.h"
#include "task_supervisor.h"

QueueHandle_t      xQueueRawData   = NULL;
QueueHandle_t      xQueueProcessed = NULL;
EventGroupHandle_t xEventAlive     = NULL;

#define STACK_ACQ   600
#define STACK_PROC  200
#define STACK_COMM  400
#define STACK_SUP   220

extern "C" void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    Serial.print(F("\r\n!!! STACK OVERFLOW in: "));
    Serial.println(pcTaskName);
    Serial.flush();
    for (;;) { ; }
}

static int freeRAM()
{
    extern char *__brkval;
    extern char  __heap_start;
    char top;
    return (int)&top - (int)(__brkval ? __brkval : &__heap_start);
}

void setup(void)
{
    Serial.begin(115200);
    while (!Serial) { ; }

    Serial.println(F("FreeRTOS - DHT11 + MQ-135 + I2C-LCD"));
    Serial.print(F("[main] RAM frei: "));
```

```
    Serial.println(freeRAM());

    acquisition_init();
    comm_init();

    xQueueRawData   = xQueueCreate(2, sizeof(sensor_data_t));
    xQueueProcessed = xQueueCreate(2, sizeof(processed_data_t));
    xEventAlive     = xEventGroupCreate();

    if (!xQueueRawData || !xQueueProcessed || !xEventAlive) {
        Serial.println(F("FATAL: Queue/Event NULL!"));
        for (;;) { ; }
    }

    BaseType_t ok;
    ok = xTaskCreate(vTaskAcquisition, "ACQ",  STACK_ACQ,  NULL, 3, NULL);
    Serial.println(ok == pdPASS ? F("[main] ACQ:  OK") : F("[main] ACQ:  FAIL!"));
    ok = xTaskCreate(vTaskProcessing,  "PROC", STACK_PROC, NULL, 2, NULL);
    Serial.println(ok == pdPASS ? F("[main] PROC: OK") : F("[main] PROC: FAIL!"));
    ok = xTaskCreate(vTaskComm,        "COMM", STACK_COMM, NULL, 1, NULL);
    Serial.println(ok == pdPASS ? F("[main] COMM: OK") : F("[main] COMM: FAIL!"));
    ok = xTaskCreate(vTaskSupervisor,  "SUP",  STACK_SUP,  NULL, 4, NULL);
    Serial.println(ok == pdPASS ? F("[main] SUP:  OK") : F("[main] SUP:  FAIL!"));

    Serial.println(F("[main] Scheduler startet..."));
    Serial.flush();
}

void loop(void) { }
```

Quellcode: Setup-Funktion zur Initialisierung der Komponenten

Name	Typ	Zweck
xQueueRawData	Queue (2 Elem.)	Rohdaten: Acquisition → Processing
xQueueProcessed	Queue (2 Elem.)	Gefilterte Daten: Processing → Communication
xEventAlive	Event Group	Alive-Signale aller Tasks → Supervisor

Tabelle 3: RTOS-Objekte

Task	Periode	Priorität	Stack (B)	Typ
Acquisition	2000 ms	3	600	Periodisch
Processing	event-driven	2	200	Event-driven
Communication	event-driven	1	400	Event-driven
Supervisor	5000 ms	4	220	Periodisch

Tabelle 4: Task-Übersicht

7. Hauptfunktionen des Codes

7.1 Task 1: Sensorerfassung (Acquisition)

Der Acquisition-Task liest alle 2 Sekunden die Temperatur und Luftfeuchtigkeit vom DHT11 sowie den Luftqualitätswert vom MQ-135. Da die DHT11-Library intern Interrupts für das Timing benötigt, wird vTaskSuspendAll() statt taskENTER_CRITICAL() verwendet. Nach jedem Messzyklus toggelt die Heartbeat-LED (Pin 7). Der Task verwendet vTaskDelayUntil() für deterministische Abtastung.

```
55  void vTaskAcquisition(void *pvParameters)
56  {
57      (void)pvParameters;
58      sensor_data_t sample;
59
60      /* — DHT11 + MQ-135 Aufwärmzeit — */
61      vTaskDelay(pdMS_TO_TICKS(2000));
62
63      /* — xLastWakeTime NACH dem Delay setzen! —
64       * Sonst versucht vTaskDelayUntil die "verpasste" Periode
65       * nachzuholen → zwei Messungen direkt hintereinander.
66       */
67      TickType_t xLastWakeTime = xTaskGetTickCount();
68
69      for (;;) {
70          float t, h;
71          uint16_t aq;
72
73          /* ═══════════════════════════════════════════════════
74           * vTaskSuspendAll() = Scheduler pausieren
75           *   → Kein Task-Wechsel während DHT-Messung
76           *   → Interrupts bleiben AN (DHT-Library braucht das!)
77           *
78           * NICHT taskENTER_CRITICAL()!
79           *   → Das sperrt Interrupts → kollidiert mit DHT-Library.
80           *
81           * MQ-135: analogRead() ist schnell (~0,1 ms) und
82           *         interrupt-unkritisch → ebenfalls im suspendierten
83           *         Zustand lesen, damit keine Kontextwechsel stören.
84           * ═══════════════════════════════════════════════════ */
85          vTaskSuspendAll();
86          t  = dht.readTemperature();
87          h  = dht.readHumidity();
88          aq = (uint16_t)analogRead(MQ135_PIN);
89          xTaskResumeAll();
90
91          /* — NaN-Retry für DHT11 (MQ-135 liefert immer einen Wert) — */
92          if (isnan(t) || isnan(h)) {
93              vTaskDelay(pdMS_TO_TICKS(200));
94
95              vTaskSuspendAll();
96              t = dht.readTemperature(false, true);   // force=true
```

```
            h = dht.readHumidity(true);             // force=true
            xTaskResumeAll();
        }

        sample.temperature  = isnan(t) ? -99.0f : t;
        sample.humidity     = isnan(h) ? -1.0f  : h;
        sample.airQuality   = aq;
        sample.timestamp_ms = millis();

        /* ---- Queue senden (neuester Wert hat Vorrang) --------------- */
        if (xQueueSend(xQueueRawData, &sample, 0) != pdTRUE) {
            sensor_data_t dummy;
            xQueueReceive(xQueueRawData, &dummy, 0);
            xQueueSend(xQueueRawData, &sample, 0);
        }

        /* ---- Alive-Signal ------------------------------------------ */
        xEventGroupSetBits(xEventAlive, EVT_ACQ_ALIVE);

        /* ---- Heartbeat-LED toggeln (sichtbar alle 2 s) -------------- */
        digitalWrite(PIN_LED_HB, !digitalRead(PIN_LED_HB));

        /* ---- Debug auf UART ----------------------------------------- */
        Serial.print(F("[ACQ] "));
        Serial.print(sample.temperature, 1);
        Serial.print(F("C "));
        Serial.print(sample.humidity, 1);
        Serial.print(F("% AQ="));
        Serial.print(sample.airQuality);
        Serial.print(F(" @"));
        Serial.println(sample.timestamp_ms);

        /* ---- Nächste Periode ---------------------------------------- */
        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(ACQ_PERIOD_MS));
    }
}
```

Quellcode: Acquisition-Task

7.2 Task 2: Datenverarbeitung (Processing)

Der Processing-Task ist event-driven und blockiert auf der Queue. Bei Empfang neuer Rohdaten berechnet er einen gleitenden Mittelwert (Moving Average) über die letzten 5

Messwerte mittels eines Ringpuffers und prüft die Schwellwerte. Die Alarm-LEDs und der Buzzer werden direkt nach der Schwellwertprüfung gesetzt.

Parameter	Schwellwert	Flag	LED	Buzzer
Temperatur	> 30.0 °C	ALERT_TEMP_HIGH (Bit 0)	Pin 8 rot	AN
Feuchtigkeit	> 80.0 %RH	ALERT_HUM_HIGH (Bit 1)	Pin 9 gelb	AN
Luftqualität	> 600 ADC	ALERT_AQ_HIGH (Bit 2)	Pin 10 blau	AN

Tabelle 5: Alarm-Schwellwerte

```
37  void vTaskProcessing(void *pvParameters)
38  {
39      (void)pvParameters;
40      sensor_data_t    raw;
41      processed_data_t proc;
42
43      /* ---- Alarm-LEDs + Buzzer initialisieren ---------------------------- */
44      pinMode(PIN_LED_TEMP, OUTPUT);  digitalWrite(PIN_LED_TEMP, LOW);
45      pinMode(PIN_LED_HUM,  OUTPUT);  digitalWrite(PIN_LED_HUM,  LOW);
46      pinMode(PIN_LED_AQ,   OUTPUT);  digitalWrite(PIN_LED_AQ,   LOW);
47      pinMode(PIN_BUZZER,   OUTPUT);  digitalWrite(PIN_BUZZER,   LOW);
48
49      for (;;) {
50          if (xQueueReceive(xQueueRawData, &raw, portMAX_DELAY) == pdTRUE) {
51
52              /* ---- Ringpuffer aktualisieren ------------------------------ */
53              tempBuf[bufIdx] = raw.temperature;
54              humBuf[bufIdx]  = raw.humidity;
55              aqBuf[bufIdx]   = (float)raw.airQuality;   // uint16 → float
56              bufIdx          = (bufIdx + 1) % FILTER_SIZE;
57              if (bufCount < FILTER_SIZE) bufCount++;
58
59              /* ---- Mittelwerte berechnen --------------------------------- */
60              proc.tempFiltered = averageFloat(tempBuf, bufCount);
61              proc.humFiltered  = averageFloat(humBuf,  bufCount);
62              proc.aqFiltered   = averageFloat(aqBuf,   bufCount);
63              proc.timestamp_ms = raw.timestamp_ms;
64
65              /* ---- Schwellwertprüfung ------------------------------------ */
66              proc.alertFlags = 0;
67
68              if (proc.tempFiltered > THRESHOLD_TEMP_HIGH)
69                  proc.alertFlags |= ALERT_TEMP_HIGH;
70
71              if (proc.humFiltered > THRESHOLD_HUM_HIGH)
72                  proc.alertFlags |= ALERT_HUM_HIGH;
73
74              /* MQ-135: hoher ADC-Wert = schlechte Luft → Alarm */
75              if (proc.aqFiltered > (float)THRESHOLD_AQ_HIGH)
76                  proc.alertFlags |= ALERT_AQ_HIGH;
```

```
77
78          /* ---- Alarm-LEDs setzen ------------------------------------ */
79          digitalWrite(PIN_LED_TEMP, (proc.alertFlags & ALERT_TEMP_HIGH) ? HIGH : LOW);
80          digitalWrite(PIN_LED_HUM,  (proc.alertFlags & ALERT_HUM_HIGH)  ? HIGH : LOW);
81          digitalWrite(PIN_LED_AQ,   (proc.alertFlags & ALERT_AQ_HIGH)   ? HIGH : LOW);
82
83          /* ---- Buzzer: tone() bei Alarm, noTone() wenn OK ------------ */
84          if (proc.alertFlags != 0) {
85              tone(PIN_BUZZER, 1000);   // 1000 Hz - passiver Buzzer
86          } else {
87              noTone(PIN_BUZZER);
88          }
89
90          /* ---- Weiterleiten ----------------------------------------- */
91          if (xQueueSend(xQueueProcessed, &proc, pdMS_TO_TICKS(50)) != pdTRUE) {
92              processed_data_t dummy;
93              xQueueReceive(xQueueProcessed, &dummy, 0);
94              xQueueSend(xQueueProcessed, &proc, 0);
95          }
96
97          /* ---- Alive-Signal ----------------------------------------- */
98          xEventGroupSetBits(xEventAlive, EVT_PROC_ALIVE);
99
100         /* ---- Debug ------------------------------------------------ */
101         if (proc.alertFlags != 0) {
102             Serial.print(F("[PROC] ALARM! Flags=0x"));
103             Serial.println(proc.alertFlags, HEX);
104         }
105     }
106 }
107 }
```

Quellcode: Processing-Task

7.3 Task 3: Kommunikation (Communication)

Der Communication-Task gibt die gefilterten Daten auf dem LCD-Display (I2C 0x27) und über UART im CSV-Format aus. Da auf AVR kein snprintf(%f) zuverlässig funktioniert, wird dtostrf() für die Float-Formatierung verwendet.

```c
void vTaskComm(void *pvParameters)
{
    (void)pvParameters;
    processed_data_t data;

    /* Puffer für dtostrf (AVR-sicher, kein snprintf(%f)!) */
    char tmpStr[7];    // z.B. " 25.3" → max 6 Zeichen + '\0'
    char humStr[7];
    char aqStr[5];     // MQ-135 ADC: 0–1023 → max 4 Zeichen + '\0'

    for (;;) {
        if (xQueueReceive(xQueueProcessed, &data, portMAX_DELAY) == pdTRUE) {

            /* == Floats → Strings (dtostrf ist AVR-sicher) == */
            dtostrf(data.tempFiltered, 4, 1, tmpStr);  // "25.3"
            dtostrf(data.humFiltered,  4, 1, humStr);  // "60.0"

            /* AQ: Mittelwert auf Integer runden → itoa reicht */
            uint16_t aqInt = (uint16_t)(data.aqFiltered + 0.5f);
            itoa(aqInt, aqStr, 10);

            /* == LCD Zeile 0: "T:25.3C H:60.0%" ==
             *     16 Zeichen: T:_25.3C_H:_60.0%
             *                 1234567890123456        */
            lcd.setCursor(0, 0);
            lcd.print(F("T:"));
            lcd.print(tmpStr);
            lcd.print(F("C H:"));
            lcd.print(humStr);
            lcd.print('%');

            /* == LCD Zeile 1: "AQ:xxxx ST:[OK] " ==
             *     oder         "AQ:xxxx ST:[ALM]"     */
            lcd.setCursor(0, 1);
            lcd.print(F("AQ:"));
```

```
102
103             /* AQ-Feld: exakt 4 Stellen (rechtsbündig mit Leerzeichen) */
104             uint8_t aqLen = strlen(aqStr);
105             for (uint8_t i = aqLen; i < 4; i++) lcd.print(' ');
106             lcd.print(aqStr);
107
108             lcd.print(F(" ST:"));
109             if (data.alertFlags != 0) {
110                 lcd.print(F("[ALM]"));
111             } else {
112                 lcd.print(F("[OK] "));
113             }
114
115             /* == UART CSV-Ausgabe == */
116             Serial.print(data.timestamp_ms);
117             Serial.print(',');
118             Serial.print(data.tempFiltered, 1);
119             Serial.print(',');
120             Serial.print(data.humFiltered, 1);
121             Serial.print(',');
122             Serial.print(aqInt);
123             Serial.print(',');
124             Serial.println(data.alertFlags, HEX);
125
126             /* ---- Alive-Signal ---------------------------------------- */
127             xEventGroupSetBits(xEventAlive, EVT_COMM_ALIVE);
128         }
129     }
130 }
131
```

Quellcode: Communication-Task

7.4 Task 4: Überwachung (Supervisor)

Der Supervisor-Task hat die höchste Priorität und läuft alle 5 Sekunden. Er prüft über die Event Group, ob alle drei anderen Tasks ihr Alive-Signal gesetzt haben. Bei fehlendem Signal leuchtet die Fehler-LED an Pin 13 auf.

```
21  void vTaskSupervisor(void *pvParameters)
22  {
23      (void)pvParameters;
24
25      pinMode(PIN_LED_ERROR, OUTPUT);
26      digitalWrite(PIN_LED_ERROR, LOW);
27      vTaskDelay(pdMS_TO_TICKS(SUP_PERIOD_MS + 500));
28
29      TickType_t xLastWakeTime = xTaskGetTickCount();
30      uint32_t   cycleCount    = 0;
31
32      for (;;) {
33          cycleCount++;
34
35          /* Bits lesen + zurücksetzen (pdTRUE), NICHT auf alle warten (pdFALSE) */
36          EventBits_t bits = xEventGroupWaitBits(
37              xEventAlive, EVT_ALL_ALIVE, pdTRUE, pdFALSE, 0);
38
39          bool allAlive = true;
40          if (!(bits & EVT_ACQ_ALIVE)) {
41              Serial.println(F("[SUP] WARN: ACQ fehlt!"));
42              allAlive = false;
43          }
44          if (!(bits & EVT_PROC_ALIVE)) {
45              Serial.println(F("[SUP] WARN: PROC fehlt!"));
46              allAlive = false;
47          }
48          if (!(bits & EVT_COMM_ALIVE)) {
49              Serial.println(F("[SUP] WARN: COMM fehlt!"));
50              allAlive = false;
51          }
52
53          digitalWrite(PIN_LED_ERROR, allAlive ? LOW : HIGH);
54
55          UBaseType_t rawUsed  = uxQueueMessagesWaiting(xQueueRawData);
56          UBaseType_t procUsed = uxQueueMessagesWaiting(xQueueProcessed);
57
58          Serial.print(F("[SUP] #"));
59          Serial.print(cycleCount);
60          Serial.print(allAlive ? F(" OK") : F(" ERR"));
61          Serial.print(F(" Q:"));
62          Serial.print(rawUsed);
63          Serial.print('/');
64          Serial.print(QUEUE_DEPTH);
65          Serial.print(',');
66          Serial.print(procUsed);
67          Serial.print('/');
68          Serial.print(QUEUE_DEPTH);
69          /* uxTaskGetStackHighWaterMark() = verbleibende Words im SUP-Stack.
70           * Wert < 20 → Stack vergrößern (STACK_SUP in main.cpp erhöhen). */
71          Serial.print(F(" SUP-HWM="));
72          Serial.print(uxTaskGetStackHighWaterMark(NULL));
73          Serial.println(F("W"));
74
75          vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(SUP_PERIOD_MS));
76      }
77  }
```

Quellcode: Supervisor-Task

8. Tests und Validierung

Nach der Implementierung des Überwachungssystems wurden umfassende Funktionstests durchgeführt, um die Zuverlässigkeit und das korrekte Zusammenspiel der verwendeten Komponenten zu gewährleisten. Dabei lag der Fokus auf der korrekten Erkennung von Sensorwerten sowie der korrekten Funktion von Alarm-LEDs, LCD und Buzzer.

8.1 Serielle Monitor-Ausgabe

Die Ausgabe auf dem Seriellen Monitor zeigt in Echtzeit, ob Sensordaten korrekt erfasst und Alarme ausgelöst werden:

- Normalbetrieb: CSV-Zeilen alle 2 s mit Timestamp, Temperatur, Feuchtigkeit, AQ und AlertFlags (0)
- Bei Schwellwertüberschreitung: AlertFlags ≠ 0, LED leuchtet, Buzzer ertönt
- [SUP] OK erscheint alle 5 Sekunden – alle Tasks aktiv
- [SUP] WARN: XY fehlt! – Task hat kein Alive-Signal gesendet

Hier die Beispielausgabe des Seriellen Monitors:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

*  Executing task: C:\Users\zakaria.rajjaa\.platformio\penv\Scripts\platformio.exe device monitor
FreeRTOS - DHT11 + MQ-135 + I2C-LCD
[main] RAM frei: 7420
[ACQ] DHT11 initialisiert (Pin 2)
[ACQ] HINWEIS: DHT11 Genauigkeit: Temp ±2°C, Hum ±5%
[ACQ] MQ-135 initialisiert (Pin A0)
[ACQ] HINWEIS: MQ-135 braucht ~24h Einbrennzeit!
[ACQ] Heartbeat-LED Pin 7
[COMM] LCD 1602 init via I2C
[COMM] I2C-Adresse: 0x27
[COMM] SDA=Pin20, SCL=Pin21 (Mega)

--- CSV DATA BEGIN ---
timestamp_ms,temp_C,hum_pct,aq_raw,alerts
[main] ACQ:  OK
[main] PROC: OK
[main] COMM: OK
[main] SUP:  OK
[main] Scheduler startet...
[ACQ] 21.3C 41.0% AQ=43 @3191
3191,21.3,41.0,43,0
[ACQ] 21.3C 41.0% AQ=45 @5218
5218,21.3,41.0,44,0
[SUP] #1 OK Q:0/2,0/2 SUP-HWM=128w
[ACQ] 21.3C 41.0% AQ=43 @7246
7246,21.3,41.0,44,0
[ACQ] 21.3C 41.0% AQ=45 @9274
9274,21.3,41.0,44,0
[ACQ] 21.3C 41.0% AQ=43 @11301
11301,21.3,41.0,44,0
[SUP] #2 OK Q:0/2,0/2 SUP-HWM=128w
[ACQ] 21.3C 41.0% AQ=46 @13327
13327,21.3,41.0,44,0
[ACQ] 21.3C 41.0% AQ=43 @15356
15356,21.3,41.0,44,0
[SUP] #3 OK Q:0/2,0/2 SUP-HWM=128w
[ACQ] 21.3C 41.0% AQ=45 @17382
17382,21.3,41.0,44,0
[ACQ] 21.3C 41.0% AQ=43 @19411
```

Quellcode: Serial Monitor Ausgabe

Alle durchgeführten Tests wurden bestanden. Die folgende Tabelle fasst die Ergebnisse zusammen:

#	Test	Methode	Erwartetes Ergebnis	Status
T1	DHT11 Temperatur	Finger auf Sensor	T steigt auf ~30 °C	Bestanden
T2	DHT11 Feuchtigkeit	Sensor anhauchen	H steigt auf ~70–80 %	Bestanden
T3	MQ-135 Luftqualität	In Sensor pusten	AQ steigt auf ~300+	Bestanden
T4	LCD-Anzeige	Normalbetrieb	Werte korrekt angezeigt	Bestanden
T5	UART CSV-Ausgabe	Serial Monitor	CSV korrekt formatiert	Bestanden
T6	Alarm-LEDs	Schwellwert überschr.	LED leuchtet sofort	Bestanden
T7	Buzzer-Alarm	Schwellwert überschr.	1000 Hz Ton aktiv	Bestanden
T8	Heartbeat-LED	Systembeobachtung	LED toggelt alle 2 s	Bestanden
T9	Supervisor	Statusmeldungen	[SUP] OK alle 5 s	Bestanden
T10	Periodizität	Timestamps vergleichen	Delta ≈ 2000 ms (±2 ms)	Bestanden

Tabelle 6: Funktionstests

8.2 Fehleranalyse und Lösungsansätze

Während der Testphase traten verschiedene Probleme auf, die systematisch analysiert und behoben wurden:

Problem: LCD zeigt nichts an

Ursache: Ursache: Das Kontrast-Potentiometer war falsch eingestellt. Bei einigen Modulen ist die I2C-Adresse 0x3F statt 0x27.

Lösung: Lösung: ✓ Kontrast-Poti anpassen. ✓ I2C-Scanner-Sketch zur Adressidentifikation verwenden.

Problem: DHT11 liefert NaN

Ursache: Ursache: Der DHT11 benötigt ~2 Sekunden Aufwärmzeit nach dem Einschalten.

Lösung: Lösung: ✓ vTaskDelay(2000 ms) am Task-Start ergänzt. ✓ Automatischer Retry-Mechanismus mit force=true implementiert.

Problem: DHT11 und taskENTER_CRITICAL()

Ursache: Ursache: taskENTER_CRITICAL() sperrt Interrupts – das zerstört das Timing der DHT11-Library.

Lösung: Lösung: ✓ vTaskSuspendAll() pausiert nur den Scheduler, Interrupts bleiben aktiv. Wichtige Erkenntnis für Sensoren mit zeitkritischen Interrupt-Signalen.

Problem: Buzzer erzeugt keinen Ton

Ursache: Ursache: digitalWrite(HIGH) auf einem passiven Buzzer erzeugt keinen Ton.

Lösung: Lösung: ✓ tone (PIN_BUZZER, 1000) erzeugt ein 1000 Hz PWM-Signal über den PN2222-Transistor.

Problem: Stack Overflow im COMM-Task

Ursache: Ursache: Die Wire/I2C-Bibliothek legt interne Puffer auf dem Stack ab.

Lösung: Lösung: ✓ Stack von 300 auf 400 Bytes erhöht – Stack Overflow behoben

Problem: Multiple Definition Linker-Fehler

Ursache: Ursache: .cpp-Dateien wurden per #include eingebunden.

Lösung: Lösung: ✓ PlatformIO kompiliert alle .cpp-Dateien automatisch – niemals per #include einbinden.

8.3 Ergebnisse und Diskussion

Nach Abschluss der Implementierung und umfassenden Testphase konnte bestätigt werden, dass das entwickelte FreeRTOS-Umgebungsüberwachungssystem alle definierten Anforderungen erfüllt und zuverlässig arbeitet.

Hauptergebnisse im Überblick:

- Zuverlässige Sensorerfassung: DHT11 und MQ-135 liefern korrekte Werte mit NaN-Schutz
- Stabiler gleitender Mittelwert: Ringpuffer über 5 Messwerte filtert Ausreißer
- Klare Benutzerführung: LCD zeigt Temperatur, Feuchtigkeit, Luftqualität und Alarmstatus
- Effektive Alarmsignalisierung: LEDs und Buzzer reagieren sofort beim Schwellwertüberschreiten
- Zuverlässige Systemüberwachung: Supervisor erkennt fehlende Tasks innerhalb von 5 s

Task	Periode	WCET	Auslastung
Acquisition	2000 ms	~18 ms	0,90 %
Processing	2000 ms	~2 ms	0,10 %
Communication	2000 ms	~30 ms	1,50 %
Supervisor	5000 ms	~2 ms	0,04 %
Gesamt	–	–	≈ 2,54 %

Tabelle 7: Deadline-Analyse und CPU-Auslastung

Mögliche Erweiterungen zur weiteren Optimierung:

- Cloud-Anbindung: Integration einer IoT-Plattform über LoRa oder MQTT für Fernüberwachung
- Datenspeicherung: SD-Karten-Integration für lokales Datenlogging
- Präzisere Sensoren: Austausch des DHT11 gegen einen SHT31-Sensor (±0,3 °C)
- Erweiterte Anzeige: OLED-Display oder Webinterface für komfortablere Visualisierung.

9. Fazit

Das Ziel dieser Arbeit war die Entwicklung eines FreeRTOS-basierten Umgebungsüberwachungssystems auf Basis eines Arduino Mega 2560. Die definierten Anforderungen konnten vollständig umgesetzt werden, wodurch eine modulare, zuverlässige und echtzeitfähige Überwachung von Temperatur, Luftfeuchtigkeit und Luftqualität gewährleistet wird.

Die Komponenten DHT11, MQ-135, LCD-Display, Status-LEDs und Buzzer arbeiten stabil zusammen und ermöglichen eine benutzerfreundliche und reaktionsschnelle Bedienung. Dabei wurden praktische Erfahrungen im Umgang mit FreeRTOS, I2C, ADC und 1-Wire-Protokoll gesammelt sowie wertvolle Kenntnisse im Bereich der Hardwareintegration und Mikrocontroller-Programmierung gewonnen.

Abschließend bietet das Projekt zahlreiche Möglichkeiten für zukünftige Erweiterungen, insbesondere durch die Integration von Cloud-Anbindungen, Datenspeicherung sowie präziserer Sensorik. Somit bildet dieses Projekt eine solide Grundlage für weiterführende Entwicklungen und Anwendungen im Bereich der eingebetteten Systeme.

Git-Repository: https://github.com/zrajjaa/Projekt_ES1

Alle Quelldateien, Diagramme und die vollständige Dokumentation (DESIGN.md, README.md) sind im Repository verfügbar.